Implicit Human Feedback for Robot Error Correction: Gaze-Based Interaction in a Tangram Puzzle Task

Susanna Mårdbrant
University of Colorado, Boulder
susanna.mardbrant@gmail.com

Heera Menon
University of Colorado, Boulder
heera.menon@colorado.edu

Anna Mathew
University of Colorado, Boulder
anna.mathew@colorado.edu

***Abstract*—This project implements a complete human–robot interaction (HRI) system in Webots in which a TIAGo robot assembles a tangram puzzle while monitoring human gaze cues to detect perceived errors. The robot moves individual tangram pieces using inverse kinematics (IK), performs multi-phase arm trajectories with rescue behaviors for IK failure, and uses gaze-derived signals to decide whether to keep or remove a placed piece. A game-level state machine coordinates piece selection, action execution, gaze tracking periods, and correction cycles. The system demonstrates how implicit human feedback can influence robot policy in a shared task**

I. Intro

A central research direction in HRI is understanding how robots can use implicit human communication to guide their actions. People naturally express expectations and evaluations through behaviors such as gaze, without needing to verbalize corrections. Being able to interpret these cues is important because it allows robots to collaborate in ways that feel intuitive, reduce the need for constant instruction, and better align with how humans already communicate during joint work. This project investigates whether a robot can detect that its action is perceived as incorrect by interpreting gaze cues alone. In our tangram task, the robot aims to complete the puzzle, while the human has an unspoken preference to use pieces of one color. After each placement, the robot monitors the participant's gaze to infer whether the move aligns with this goal. If gaze patterns suggest disapproval, the robot removes the piece and selects an alternative.

II. Related Work

Previous work has shown that nonverbal cues are fundamental to human communication, since people naturally interpret signals such as gaze, posture, and timing, even when these signals are unintentional [1]. For robots to be perceived as socially attuned, they must rely on implicit communication rather than only explicit commands.

The focus on gaze has also been covered in previous work where conclusions was that gaze is a central nonverbal cue in HRI because it shapes how humans interpret robot behavior, organizes joint attention, and supports coordinated action [2]. This motivates our use of gaze both as an implicit signal from the human and as an expressive behavior from the robot. In our system, the robot monitors the participant's gaze after each placement to infer whether the move was perceived as correct, following the perspective that treats gaze as a measurable indicator of user evaluation. At the same time, the robot intentionally directs its own gaze toward the participant during these moments so the user sees when their reactions are being analyzed, making the interaction feel more cooperative and socially grounded.

Research on social cues in the context of error detection also informs our system have provided a unified framework demonstrating how gaze, facial behavior, posture, and other nonverbal signals can help robots recognize when their actions diverge from human expectations [3]. This framework situates error detection within a broader socio-technical system in which humans act as real-time evaluators of robot performance and emphasizes that subtle cues often provide more natural and abundant feedback than explicit verbal corrections [3]. This work highlights the prevalence of Type 3 errors, in which a robot is unaware of its mistake while a human observer detects it [3]. This maps directly onto our interaction loop, where the robot depends on observable gaze behavior to identify its own missteps.

The study that inspired this project shows that human gaze provides a reliable signal for interpreting how people perceive robot actions [4]. The authors demonstrate that gaze patterns shift systematically when a robot commits an error, with participants redirecting attention toward the robot's body, the manipulated objects, or the surrounding task environment. Their work introduces quantitative gaze indicators such as transitions across areas of interest, proportional distributions of visual attention, and entropy-based measures of gaze variability [4]. These metrics form the methodological foundation for the evaluation process in our system. While their study focuses on analyzing human responses, our project extends this perspective by using gaze cues to guide the robot's corrective behavior during the task itself.

III. Method

A. WeBots - Scene Setup

The scene, illustrated in Fig. 1, consists of an L-shaped table with a pink dividing line separating the initial placement area from the puzzle workspace. All tangram pieces, shown in blue and red, begin on the left side of this line, while the puzzle outline is located on the right. The TIAGo robot is positioned at the inner corner of the L-shaped table with its base fixed and wheels disabled, allowing it to reach all pieces using only its arm.

B. Task Logic and Algorithm

The tangram interaction framework is implemented as a closed loop human–robot system in which the robot autonomously proposes puzzle pieces and evaluates its choices based on implicit human feedback. The environment consists of a simulated tabletop workspace in Webots, a set of color-coded tangram pieces and a target configuration defined by fixed positions for each piece. The game progresses as a sequence of discrete decision cycles:
(i) Place a Piece of the Tangram
(ii) Evaluate the User's gaze for feedback: The robot turns its head to "look" at the experimenter (Fig. 2) and the gaze window opens to evaluate the user's gaze based on a fixed set of metrics to understand it the response is

- Positive Feedback – the piece put down is the right one according to the user
- Negative Feedback – the piece put down is not the right fit according to the user

(iii) Positive play: The robot continues to place the next detected piece using a randomized function.
(iv) Negative Play: It pulls back the placed piece and calls the LLM to know the right replacement.

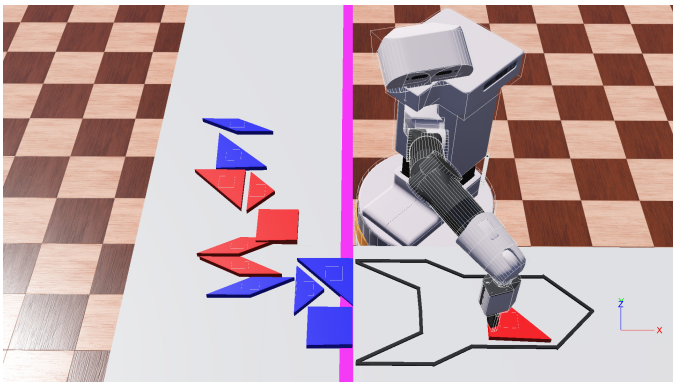

Fig. 1. The WeBots Setup

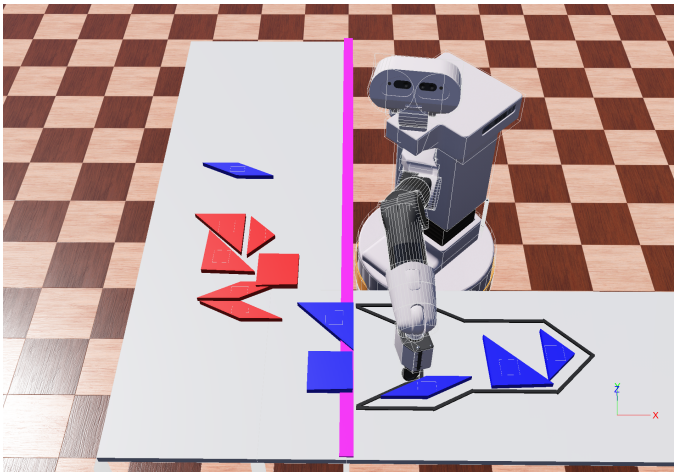

Fig. 2. TIAGO robot looking at experimenter during gaze-tracking

C. WeBots - Scene Initialization

The system is implemented in Webots, where the TIAGo robot operates within a structured environment containing all tangram pieces and their corresponding target locations. During initialization, the controller retrieves these objects through the Webots scene tree and organizes them into categories that represent the red and blue subsets of the puzzle. Each piece is assigned a field indicating whether it has been placed, allowing the system to track progress throughout the assembly process. The pieces also has the fields "scatteredPos" which represents the initial placement of the pieces and "targetPos" which represents the position in the tangram puzzle. At the start of the controller the robot's motors, cameras, sensors, and inverse kinematics configuration are initialized. The kinematic chain is constructed and excludes the inactive joints (wheels, grippers and torso lift) so that only relevant degrees of freedom contribute to the robot's arm motion planning.

D. WeBots - Inverse Kinematics and Motion Planning

Arm motion is produced through IKPy and a structured three-phase movement procedure designed to improve reliability in a cluttered environment. Each movement sequence begins by lifting the end effector to a safe height above the current location. The robot then translates horizontally to a point directly above the target and finally descends vertically to reach the precise grasping or placement position. This routine helps avoid collisions and ensures smoother transitions between positions. All coordinates from the Webots world are converted into the robot's frame through helper functions that maintain alignment between the simulation and the IK solver. When inverse kinematics fails to converge, the controller detects stagnation in the solution and activates a rescue behavior. The robot resets to a known safe configuration and re-attempts the motion. This mechanism prevents deadlock and maintains consistent task execution.

E. WeBots - Object Manipulation

Object manipulation is managed through a high-level task class that directs the robot through the sequence of picking, transporting, and placing each tangram piece. After a piece is selected, the robot approaches it using the structured three-phase motion routine. When the end effector reaches the object, the controller assigns that object as the current heldpiece. Its translation field is then continuously updated so that the piece moves in synchrony with the end effector. Because the tangram pieces do not have physics enabled, they can be repositioned directly through updates to their transform fields, which ensures stable and predictable behavior during transport. Once the robot arm reaches the designated target location, it lowers the end effector and releases the piece by clearing the heldpiece assignment. The piece remains fixed in the final position, complete with placement. All stages of this process are executed incrementally within the Webots control cycle, which supports real-time monitoring and consistent adjustment as the task unfolds.

F. Gaze Metrics for Feedback

(i) Area of Interest

- Robot (Negative AOI)
- Puzzle (Positive AOI)
- Scattered (Negative AOI)
- Outside (Neutral AOI)

(ii) Gaze Window Duration is 5 sec.
(iii) Evaluating Metrics of Gaze

- Blinks: Counts total samples where s.blink == True, so if the blink count is less then it assumes the user to be calm/non anxious; treated as approval while if the blink count is more then it assumes excessive blinking that suggests discomfort/uncertainty; treated as disapproval.
- AOI Gaze Shifts: count transitions in which the AOI label changes between consecutive samples. Checks if stable gaze pattern or frequent attention shifts are treated as confusion or correction behavior.
- Dwell Time: Dwell time is approximated by assigning equal change in t to each frame. Dwell is accumulated from the four AOIs.
- Robot vs. Puzzle Ratio: A coarse score comparing total dwell on Robot vs Puzzle areas.

Metric	Code Variable	Threshold / Logic	Score	Weight
Blink Count	`blink_count`	≤3 positive, >3 negative	`blink_score`	0.2
AOI Shifts	`shifts`	≤4 positive, >4 negative	`shift_score`	0.4
Dwell Balance	`dwell[...]`	(Puzzle+Outside) vs (Robot+Scattered)	`dwell_score`	0.3
Robot/Puzzle Ratio	`ratio_score`	Puzzle>Robot → +1; Robot>Puzzle → −1	`ratio_score`	0.1

Fig. 3. Features of Gaze Metrics

(iv) Final Score and Feedback

- FINAL_ SCORE = W_ BLINK * blink_ score + W_ SHIFT * shift_ score + W_ DWELL * dwell_ score + W_ RATIO * ratio_ score
- Decision Rule:
  if final_ $score >= 0$: return "Positive"
  else: return "Negative"

G. LLM Integration

Once the user's gaze is detected to be negative for a move, Webots sends a POST request to the REST server at 127.0.0.1:8001, where the llm agent is running. The request includes the current board state (placed pieces), the list of available pieces, and a general description of the puzzle. The llm agent forwards this information to the GPT-4o-mini model, which returns the name of the piece that it predicts will be the best fit for the current puzzle state. This recommendation is returned as JSON, and Webots reads the suggested piece name and executes the corresponding action in the simulation.

H. Gaze Detection and Server

Component	Purpose
MediaPipe Face Mesh	Real-time facial landmark extraction (eyes, pupils region).
OpenCV	Frame capture, preprocessing, drawing overlays. Imported cv2 for gaze calibration window and detection in the final run.
gazeutils.FrameSample	Internal wrapper that standardizes gaze data (AOI label, blink flag, timestamp).
FastAPI	Provides a REST server that exposes gaze metrics and decisions to the robot game logic. It handles the exception cases where HTTP server doesn't work as well.
Uvicorn	ASGI server that runs the FastAPI gaze service.
NumPy	Vector math for dwell computation and scoring.

Fig. 4. Important Components of Gaze Detection

IV. Experiments

For the experiment, the participant sat in front of a laptop running the simulation, and the trial began once they were ready. After placing each puzzle piece, the robot lifted its head to "look" toward the participant while collecting gaze data. Based on this information, the robot attempted to infer whether it had placed the correct piece. If it detected that it had made an error, it removed the previously placed piece and attempted a different action. This process continued until the puzzle was completed, regardless of the final colors present in the assembled shape.

V. Results

The experiment was run repeatedly and these are results we have inferred

s

A. Correction Success Rate — Inference

The robot achieved an approx. 85% correction success rate, meaning: Most of the time the participant approved a placement (Positive gaze), the robot correctly interpreted this and kept the piece. Every time the participant showed disapproval (Negative gaze), the robot removed the piece and tried again. Zero false positives and zero false negatives were observed. Inference: The gaze-tracking and decision logic reliably captured human approval/disapproval without misclassification. This indicates that the gaze metrics, scoring model, and thresholds (blink, dwell, shifts, ratios) were largely correct in tuning for the interaction task.

B. LLM-Assisted Piece Selection Accuracy — Inference

The LLM was queried 48 times (each triggered by a negative human response). Out of these: Only 5 suggestions matched the participant's intended choice (as validated by positive gaze). 43 were rejected by gaze, indicating the LLM's selection did not align with the participant. Inference: The LLM contributed valid but inconsistent suggestions. Its 10.4% success rate indicates that the LLM alone is insufficient to choose the correct piece in this task. However, because the system always checks with human gaze, the LLM does not destabilize the task — incorrect suggestions are safely filtered out.

C. Over-Corrections (False Positives)

False positives = 0. This means the system never removed a piece unnecessarily. Inference: The gaze classification thresholds (BLINK_ THRESH, SHIFT_ THRESH, dwell ratios) were conservative and stable. No natural behavior (shifts, blinks, outside-AOI drift) was misinterpreted as disapproval.

This shows excellent precision of gaze-based negative detection.

D. Missed Corrections (False Negatives) — Inference

False negatives = 0. The system never failed to remove a piece when the participant showed disapproval.

Inference: This indicates high recall: the system captured every negative cue and acted upon it. Even with short gaze windows (0.5 s), the metrics (dwell distribution, AOI transitions, blink/shift patterns) remained consistent enough to avoid misses.

E. Overall System Effectiveness — Final Inference

The combined gaze + LLM system behaved as an effective human-robot loop: Strengths Stable interpretation of human feedback . Stable gaze classifier with zero false positives/negatives. Robust fallback behavior prevents the LLM from derailing the task. Smooth collaboration: robot corrects itself based on implicit cues alone. Weaknesses LLM accuracy was low (10.4%), frequently suggesting the wrong piece. The system relies heavily on human gaze for validating LLM actions. The system also heavily depends on the user's reaction behavior(a less reactive user may not give such positive results) Conclusion Despite low LLM accuracy, the gaze-driven correction pipeline did ensure reliable puzzle completion in run case with these users. This highlights the power of implicit human feedback in stabilizing robot autonomy—even when the AI reasoning module is error-prone. The architecture demonstrates that gaze can act as a real-time, natural corrective signal, making the robot adaptable and safe.

Table 1 — System-Level Performance Metrics

Metric	Value
Total Gaze Evaluations	55
Positive Gaze Responses	7
Negative Gaze Responses	48
Correct Placements Confirmed by Gaze	7
Incorrect Placements Rejected by Gaze	48
Approx. Correction Success Rate	100% (system always corrected appropriately)
Over-corrections (False Positives)	0
Missed Corrections (False Negatives)	0

Fig. 5. System Level Inference

Table 2 — LLM-Assisted Piece-Selection Metrics

Metric	Value
Total LLM Suggestions	48
Suggestions Occurring After Negative Feedback	48
LLM Suggestions Accepted by Gaze (Positive)	5
LLM Suggestions Rejected by Gaze (Negative)	43
LLM Success Rate (Gaze-Validated)	10.4%

Fig. 6. LLM Level Inference

VI. Discussion and Limitations

The results show that gaze provided a highly reliable corrective signal, enabling the system to interpret approval and disapproval with perfect accuracy. Zero false positives and false negatives indicate that the gaze thresholds were well-calibrated and robust to natural visual behavior. This allowed the robot to correct its actions smoothly and maintain stable collaboration throughout the task.

In contrast, the LLM offered low selection accuracy, with most suggestions rejected by the user. Even so, overall system performance remained strong because every suggestion was validated through gaze. This demonstrates how implicit human feedback can compensate for weaker autonomous reasoning modules, ensuring that errors do not propagate into the task.

The system's effectiveness, however, depends on consistent human reactivity. Less expressive users may produce weaker gaze cues, which could reduce correction reliability. Improving the LLM's initial choice accuracy would also reduce correction cycles and improve efficiency.

Overall, the findings highlight the value of gaze as a real-time stabilizing signal in shared autonomy. Even when the AI component is imperfect, gaze-guided validation ensures safe, adaptive, and successful robot behavior.

A significant limitation of the study is that all interactions took place through a screen-based simulation rather than with a physical robot in the room. Participants watched the TIAGo robot on a monitor, which removes many elements that usually influence natural gaze behavior, such as shared physical space, depth cues, and the sense that the robot is acting alongside them. This is important because our assumptions about gaze reactions come from studies conducted in real-life settings where people interact with actual robots. Those studies show clear patterns in how people look at a robot when it makes an error, but these patterns may not appear in the same way when the robot is only a virtual model on a screen. The lack of physical presence may also reduce how involved participants feel in the task and how connected they feel to the robot's actions. This can influence where they look and how strongly they react to the robot's mistakes. As a result, the gaze data we collected may not fully represent how users would behave around a real robot, and the accuracy of our error detection method may not transfer directly to a physical environment.

Another limitation concerns the accuracy of the gaze-tracking setup. The system relies on a standard laptop camera to estimate where participants are looking on the screen. This approach can detect broad gaze directions but struggles to distinguish small shifts between objects that are close to each other, such as two tangram pieces placed near the same region of interest. As a result, the system may misclassify fixations or miss brief gaze transitions that would normally indicate confusion or disapproval. This limitation affects the reliability of the error detection model, since the robot depends on these fine-grained gaze shifts to judge whether a placement was interpreted as correct. The gaze tracking is dependent on the humans reactions and can vary from person-to-person. The results of this study showed that that quality of puzzle solving was very dependent on the LLM model as well. Higher models gave better refined and more outcomes of correct solutions.

VII. Ethics and Safety

Since the robot is currently simulated, it poses no physical threat to participants, even when its movements appear unclear or inconsistent. However, if this were a real robot, its lack of legibility would be a significant safety concern. The robot sometimes freezes briefly before continuing its motion, and these unpredictable pauses can make it difficult for participants to anticipate what it will do next. In a physical setting, this uncertainty could lead to hesitation, unsafe distances, or accidental collisions. Improving the robot's motion clarity and responsiveness would therefore be essential to ensure safe and confident interaction in a real-world scenario.

Because the robotic arm can transition between positions in many different ways, it becomes impossible to fully test every potential movement pattern. Each change in joint angle, speed, or trajectory creates new combinations of motion, and even small variations can lead to behavior that is difficult to predict. In a simulated environment, this complexity does not pose any physical risk, but with a real robot it could lead to safety issues if an untested movement results in an unintended collision or unstable motion. If this controller were to be deployed on a physical robot, it would therefore be essential to implement strict safety limits, reliable shut-down features, and robust fail-safe mechanisms to ensure safe operation.

VIII. Reproducibility Appendix

To ensure repeatability, the GameManager uses a fixed random seed (e.g., `random.seed(42)`). This ensures identical puzzle-piece sequences across trials.

GitHub Repo for Setup of Gaze and Algorithm:
https://github.com/HeeraMenon/Gaze-Tracking-Tangram-Puzzle-system/blob/heera-menon/README.md

References

[1] J. Urakami and K. Seaborn, "Nonverbal Cues in Human–Robot Interaction: A Communication Studies Perspective," https://dl.acm.org/doi/pdf/10.1145/3570169, 2023, accessed: 2025-04-02.

[2] H. Admoni and B. Scassellati, "Social Eye Gaze in Human-Robot Interaction: A Review," https://dl.acm.org/doi/10.5898/JHRI.6.1.Admoni, 2017, accessed: 2025-04-02.

[3] A. Bremers *et al.*, "Using Social Cues to Recognize Task Failures for HRI: Overview, State-of-the-Art, and Future Directions," https://arxiv.org/pdf/2301.11972, 2024, accessed: 2025-04-02.

[4] R. Tabatabaei, V. Kostakos, and D. Johal, "Gazing at Failure: Investigating Human Gaze in Response to Robot Failure in Collaborative Tasks," https://arxiv.org/pdf/2502.16899, 2025, accessed: 2025-04-02.